亿信华辰
ESENSOFT

20周年客户大会 · 主题分享
2006—2026 / 20 YEARS OF ESENSOFT

金融AI Agent

工程化应用：从技术赋能到价值重构

当模型开始行动，金融真正要建设的是什么？

EsXCode × EsXWork

亿信华辰自研 · AI研发应用平台 × AI办公平台

演讲人

亿信华辰解决方案专家 吴思全

2026.10.30 · 北京亿信华辰软件有限责任公司

20
YEARS

AI会回答之后，怎么把Agent真正做进生产?

把业务语义、系统行动、验证责任组织起来，才算把一个Agent真正做好。

今天先问一个问题

怎样才算做好一个Agent?

能回答，不代表能交付。
真正要补的是三段工程：

业务语义？系统行动？
验证责任？

如何做好一个Agent：补齐三段工程

01 / 业务上下文

目标、口径、规则与历史，是否被正确装配?

上下文不对，越流畅越可能偏离。

02 / 系统行动

能在授权边界内进系统、用工具吗?

只聊天不调用工具，就进不了真实流程。

03 / 验证责任

错误能否被发现、停止、解释、回退和追责?

高责任场景里，没有证据的自动，不是智能。

本次分享主线： 第一性原理 ⟶ 工程化体系 ⟶ 成熟度与实践

工程化视角：两条价值重构实践

先承认三条事实：模型天然不是一个确定性系统

这不是某一家模型的问题，而是大模型进入生产时共同面对的工程约束。

FACT 01

AI 无状态

Stateless

每次推理都不会天然保留上一次
对话的中间状态。

它更像一次“短期记忆”：
上下文给什么，它就基于什么作答。



FACT 02

上下文窗口有限

Context Window Limit

不是把所有资料都塞给模型，
而是在每一步装配恰当信息。

信息太少，答案没有依据；
信息太多，关键约束反而被稀释。



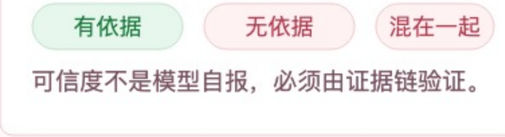
FACT 03

幻觉不可消除

Hallucination

模型会生成看似合理、
却没有依据的答案。

所以金融不能靠一句
“请确保准确”来兜底。



模型能力 × 工程确定性 = 可进入生产的AI

把概率能力变成可托付的生产力

工程化：把不确定性关进可运营的确定性外壳

不是把模型包一层界面，而是把任务、上下文、工具、状态、权限与证据组织成可重复交付的系统。

工程化 = 约束 × 运行 × 验证 × 演化 → 可重复交付　　四类体系：边界、状态、证据、版本；标准不是定义。

从“会回答”到“会交付”

普通AI应用

Prompt → Model → Output

特点：一次性输出，没有独立目标，也没有任务闭环。

Agent工程化应用

目标 → Loop / Tools → Evidence

多步推进 · 有中间状态 · 每步可验证

工程化不是“加功能”

而是把约束、验证与责任，提前设计进运行结构。

AGENT ENGINEERING STACK

业务任务层　可验收目标 · 权限边界 · 停止条件

HARNESS / RUNTIME　承载Agent Loop，组织运行状态

上下文	Agent Loop	工具 / MCP	状态 / 记忆	模型路由
装配 / 压缩	计划 / 验证	系统连接	会话 / 恢复	适配 / 替换

CONTROL PLANE / 证据与责任　让AI能停止、回退、解释、追责

权限 / 沙箱	人工确认	评测 / 回归	观测 / 审计	版本 / 回退
最小授权	关键门禁	质量基线	调用证据	产物恢复

工程化的四个结果

不是演示，而是默认值

1. **可重复** — 同一任务下次仍能跑通
2. **可解释** — 答案、动作与依据可追溯
3. **可恢复** — 出错能停，版本能退
4. **可接手** — 人换岗，任务和证据仍完整

不只金融需要

所有严肃的Agent规模化应用，都需要工程化；金融只是把准确、可靠、可追溯推到最高。

模型能力 × 工程化水平 = 可托付的Agent产能　　从加法，到乘法

Agent运行时，正在成为新的工程化底座

DeepSeek Harness 给出的信号：不是模型更强，而是运行、扩展与治理被重新组织。

PUBLIC SIGNAL · 2026.10

DeepSeek Harness

Everything is a Plugin.

它是Agent运行时/Harness，
不是一个更大的模型。

官方公开的工程事实

- 模型适配器、工具注册表、会话日志，甚至Agent Loop本身都是插件
- profile / bundle / patch 分层组合
- 能力seam连接接口、提供方与消费者
- 无特权内核，挂载插件即可扩展

开源状态：developer preview
MIT · TypeScript · 仍会存在破坏性变更

HARNESS COMPOSITION / 公开架构抽象

PROFILES web | headless | sdk | acp — 按需组合

PLUGIN TREE / 每一块能力都可挂载、替换 — no privileged core

模型适配器 Model adapter	**工具注册表** Tool registry	**会话日志** Append-only session log
Agent Loop 默认驱动器	**沙箱与审批** Sandbox / approval	**UI · 遥测 · 扩展** Product surfaces

配置组合、补丁覆盖、卸载可逆

CORDIS / 共享上下文 · 类型化事件 · 可逆副作用

插件向共享上下文贡献服务；通过事件形成扩展点；卸载时注册效果可撤销。

Service Definition | Service Provider | Consumer = Capability seam

对金融的四个启示

平台看接口，不只功能。

01 / 运行外壳
模型之外，是否有可控的运行环境?

02 / 能力接口
模型、工具、策略能否替换而不推倒重来?

03 / 证据默认
会话、动作、审批、回退是否天然留痕?

04 / 迁移壁垒
能力沉淀在接口与插件，越用越难替换。

生态竞争的下半场
比较的是接口、组合与治理。

Harness不是“多一个Agent框架”，而是把Agent运行所需的上下文、工具、状态、权限与恢复，做成可组合的工程外壳。

公开来源：DeepSeek Harness官方仓库与架构文档（developer preview，截至2026.10）；仅作行业工程信号与公开能力观察，不表示EsXCode/EsXWork采用同一架构。

06 / VALUE SHIFT

价值重构：不是更快，而是价值方程变了

技术赋能让某个环节更快；价值重构改变价值单位、成本结构、稀缺能力与能力资产。

01 / VALUE UNIT

价值单位

人天 → 可验收结果

- 功能交付、人天计费
- 结果可验收、过程可追溯
- 责任边界清晰
- 不再只数“做了多少”

先定义“完成”，再谈自动化

02 / COST STRUCTURE

成本结构

重复执行 → 工程化产能

- 人力检索、复制、核对
- 上下文装配、工具调用
- 验证、回滚、审计内建
- 生成越便宜，验证越值钱

优化链条，而不是堆人

03 / SCARCITY

稀缺能力

会写会查 → 会定义会判断

- 写材料、写代码、查资料
- 定义问题、设定边界
- 设计验收与熔断规则
- 判断例外、承担责任

人的价值向判断与责任上移

04 / ASSET FORM

能力资产

文档 → 可复用资产

- 文档、经验、个人技巧
- 上下文、技能、评测集
- 证据包、连接器、门禁
- 一次交付，多次复用

组织记忆成为生产力

VALUE EQUATION / 价值方程



不是让同一件事更快，而是让价值可以重复创造

效率红利是一次性的；
上下文、技能、证据与连接器会持续复利。

对金融的含义： 更快的代码、报表和材料只是技术赋能；能持续交付、可追溯、可审计、可复用，才是价值重构。

这也是后文判断 EsXCode 与 EsXWork 是否真正有效的标准。

价值重构不是否定效率，而是把效率从一次性收益，变成可持续的组织能力。

AI Agent成熟度三阶：工具位、协同位、目标位

升级的是AI的目标、控制权、责任边界与价值衡量。

ASSISTED

辅助型 AI

AI是工具；人是决策主体与流程主导者

AI负责

检索 · 摘要 · 翻译 · 公式计算
格式校验 · 初稿生成

控制权

人发起、人审核、人最终决策；
AI输出只作参考。

典型例子

RAG问答 · 报表初稿 · OCR提取
代码补全 · 会议纪要

阶段特征

单向输出、无闭环、
没有独立目标。

COLLABORATIVE / CO-PILOT

协同型 AI

AI是对等协作者；人机双向交互、分工完成

AI能主动

提子方案 · 调工具 · 发现异常
追问信息 · 阶段性汇报

控制权

人保留全局目标审批与关键节点拦截；
AI自主执行子任务。

典型例子

带Agent Loop的监管报送智能体
数据治理核查智能体

阶段特征

有循环、有工具、有中间状态，
但不脱离人的关键节点审批。

AUTONOMOUS AGENT

自主型 AI

完整目标闭环；在预设约束与权限边界内自主

AI能独立

拆解目标 · 规划 · 执行 · 自校验 · 重试
多子Agent协同，持续完成任务

控制权

人负责事前目标、规则边界与熔断策略；
仅在异常阈值触发时上报人。

典型形态

长任务监测预警 · 跨系统异常闭环
多Agent持续协作 · 动态重规划

金融重点：约束下自主

自主 ≠ 无监管、无审计、无熔断。
是长任务闭环、自我反思、动态规划，
不是脱离人的黑盒运行。

工具位 — AI承担的目标、控制权与责任边界 → — 目标位

请问各位： 贵机构现在把AI放在工具位、协同位，还是目标位？不用回答，带着这个判断继续往下看。

头部机构走到哪里：AI已进流程，但自主带着治理边界

只采用机构公告、年报刊载或高可信公开报道中的“已部署 / 已上线 / 公开试点”表述。

辅助型 AI｜个人与岗位提效

Morgan Stanley

Assistant + Debrief

财富顾问会前检索、会后总结；
AI出草稿，顾问确认。

BBVA

ChatGPT Enterprise

扩展至11,000席，用于内容、
分析与日常办公。

阶段特征

工作入口变了，责任链没变。
价值先在个人效率体现。

公开部署口径，不代表全流程自动化。

协同型 AI｜进入业务流程

JPMorgan Chase

LLM Suite

公开报道梳理450+生成式AI用例，
覆盖写作、分析、编码等场景。

中国工商银行

工银智涌

公开报道援引年报：已在30余个业务
领域落地500余个AI场景。

阶段特征

AI嵌入流程节点，与人共同完成。
价值转向周期、质量与一致性。

规模大，但仍以人机协同为主。

自主型方向｜公开试点与受控推进

Citi

Stylus Workspaces with Agentic AI

将多个企业级API封装成代理式工作空间，
公开口径为受控试点，不是脱离人的全自动。

平安租赁

AI数字助手 + 租赁智能体平台

已建成大模型平台，覆盖销售、风控、
运营；60余个助手，调用超百万次。

阶段特征

已进入多步任务，但并非完整目标闭环。
公开边界：试点/受控部署，不是脱离人的全自动。

自主程度取决于流程闭环与治理成熟度。

对租赁与证券的启示： 先选高价值场景，把知识、数据、系统、验证四层补齐；不要用“自主”包装尚未闭环的业务。

公开资料整理，截至2026.10；来源含Morgan Stanley、BBVA、Citi公告/新闻稿，Tearsheet与中国日报网报道；仅作行业观察，不构成效果承诺。

每个阶段，建设的不是同一个东西

Agent不是买一个模型；从辅助走向自主，要同时补场景、上下文、运行连接、证据治理与组织运营能力。

建设维度	辅助型	协同型	自主型
业务场景 从哪里开始	知识助手、文档摘要 高频、低风险、可人工复核	业务流程中的一个节点 风险、投研、合同、研发	跨步骤任务、异常监测、自动闭环 目标明确，但路径可编排
数据与上下文 答案凭什么可信	企业知识库、文档权限 有出处、可检索	数据产品、语义层、状态 业务口径可解释	实时状态、记忆、版本、知识更新 运行中不丢上下文
系统连接 能不能真正办事	轻量API、单点工具 读多写少	系统API、MCP、工作流 受控写入与回退	多工具组合、任务编排、事件触发 权限、沙箱、停止条件
验证与治理 错了怎么办	人工复核、基础审计 先保证可见	Evals、回归、人工卡点 质量可度量	可观测、停止、回退、审计、责任边界 异常可解释、可接管
组织能力 谁来持续运营	个人采用、工具培训 从会用开始	业务+科技+风险共创 产品经理与AI Ops	AI治理委员会、运营团队、定期重估边界 自主边界随成熟度移动

现场提问

在座各位，贵机构现在处在哪一阶段？下一阶段最缺的，是场景、数据、系统，还是验证？

阶段判断不看“买了什么模型”，而看：AI在哪个流程里、承担什么动作、错了谁能在多长时间内发现并接管。

两条价值重构路径：系统建设与AI办公

技术赋能让局部更快；价值重构，改变系统怎么改、工作怎么变。

不是两个产品介绍，而是两类价值重构的工程回答

系统怎么改?

存量系统二开、系统改造、新系统建设；要接上下文、控影响、留证据。

工作怎么变?

知识检索、内容生成、数据分析、流程审批；要把任务做完、交清、可追溯。

EsXCode｜面向研发的AI应用平台

自研 · 按Agent工程化原则设计 · 连接现有系统

需求上下文 → 原型/设计 → 系统变更

影响分析 → 自动测试 → 人审发布/回退

核心：接入、理解、变更、验证、证据

EsXWork｜面向办公的AI应用平台

自研 · 按Agent工程化原则设计 · 任务与专业交付

知识 / 数据 → 任务拆解 → 文档 / 表格 / PPT

连接器 / 多Agent → 审批卡点 → 归档留痕

核心：知识、任务、流程、协作、责任

它们在成熟度模型中的位置：不是一个标签，而是一组能力载体

辅助型 — **EsXCode / EsXWork 的主战场：协同型AI** — 自主型

同时具备Agent Loop、工具调用、状态与自动执行等自主型要素；但能否真正自主，仍取决于业务闭环、数据、权限与治理成熟度。

过去做系统：慢的不是代码，而是长链上的人与证据

新建系统与存量改造都必须先立项；真正的交付，是把需求、架构、接口、数据、测试和上线责任串起来。

ENTRY A
新建系统
新业务、新平台、替换旧系统

ENTRY B
存量改造 / 二开
新增功能、接口、数据或报表

两条入口，最后都要穿过同一条治理线
AI不会取消立项、预算、安全评审、验收与审计责任。

需求 / 立项 | 预算 / 采购 | 方案 / 安全 | 上线 / 验收 / 责任

立项之后，交付仍是一条串行链

每一段都依赖上一段的文档、决定与人

01 **需求与方案**
范围、边界、验收标准
人写、人讲、人对齐

02 **设计与排期**
架构、接口、数据模型
依赖资深人员判断

03 **开发与配置**
代码、二开、接口、报表
手工产出与联调

04 **集成与测试**
环境、数据、回归、缺陷
上线前集中暴露

05 **UAT 与安全**
业务确认、风险复核
签字前再补证据

06 **上线与移交**
发布、运维、知识交接
结果才第一次可检验

过去最贵的，不只是写代码，而是三件“无法自动交接”的事

01 / 上下文靠人带
需求为什么这样定、历史为什么这样改，
往往只在会议、文档与个人经验里。
新成员接手的不是系统，而是重新问一遍所有人。

02 / 影响靠经验判断
改一个字段、一段逻辑，可能牵动接口、
报表、校验与下游系统。
“改A坏B”往往在集成测试甚至上线后才被发现。

03 / 证据上线前补
测试、评审、发布说明和审计材料
常被当成项目尾声的“配套工作”。
结果能上线，却很难解释它为什么可信。

AI真正要改造的，不是某一段编码，而是这条“上下文—变更—验证—交付”的长链。

AI把“写出来”变快，把“证明可以上线”推到台前

AI可以接入授权范围内的代码、文档、接口与数据；但进入真实系统后，主战场转向“接得进、改得对、证得明”。

同一条系统建设流水线，速度结构变了

速度是局部改善，风险却在系统边界集中

01 / CONTEXT	02 / GENERATE	03 / INTEGRATE	04 / VERIFY	05 / RELEASE
理解任务	**生成候选产物**	**接进真实系统**	**证明改得对**	**签字、发布、回退**
接入需求、规范与系统现状 形成可执行上下文	需求稿、原型、代码、SQL 测试用例与交付文档	现有模块、接口、数据库 权限、环境与数据边界	影响分析、自动测试、回归 安全审查与证据包	责任确认、灰度与运行验证 知识移交与持续演进
AI加速 · 人定义边界	AI加速 · 版本可控	AI辅助 · 主瓶颈	AI加速 · 主瓶颈	人负责 · 必须可回退

瓶颈从“做出来”转移到“接进系统、验证影响、证明可信”

接得进

在客户授权边界内理解并连接：

代码仓库 · 系统模块 · API/MCP
数据字典 · 业务规则 · 现有文档

服务新建系统，也服务存量系统二开。

不是把原系统替换掉，而是把AI接进演进过程。

改得对

先回答业务与系统的影响：

哪些接口、表、报表、规则会受影响?
哪些变更可以自动，哪些必须评审?

变更范围与回退路径先于代码产生。

减少“改一处、坏一片”的隐性成本。

证得明

每次交付都能回到同一组证据：

需求依据 · 变更差异 · 自动测试
人工确认 · 版本与回退包 · 审计留痕

上线不是一个“结果”，而是一份可复核的交付。

让速度提升不牺牲可解释与可追溯。

AI研发的价值，不是让代码更多，而是让每一次系统变更更快被验证、被解释、被接手。

EsXCode：把AI放进系统工程，而不是放在聊天框

在授权边界内接入现有系统与工程上下文，把需求、影响、变更、测试、提测与回退连成一条可验证的交付链。

INPUT / CONTEXT

真实工程上下文

AI先理解系统，再开始改系统

现有系统与模块
核心、外围、数据平台

代码仓库与历史
代码图谱、提交与版本

接口、模型与数据
API/MCP、Schema、映射

规范、文档与规则
需求、设计、验收、安全

运行与测试环境
沙箱、构建、提测链接

服务对象
新系统建设 · 存量二开

SIX-STEP ENGINEERING LOOP

每次产物都有版本与证据

六步闭环：从可理解的上下文，到可回退的交付

01 / CONNECT & UNDERSTAND
接入与理解
代码/知识图谱 · 需求与系统现状
产出：任务上下文与基线

02 / PLAN & IMPACT
计划与影响分析
调用链 · 表/接口/报表影响面
产出：变更计划与风险清单

03 / SHAPE & DESIGN
原型与方案
可运行原型 · 接口/模型草案
产出：可评审的设计对象

04 / CHANGE
变更生成
代码 · SQL · 配置 · 测试草案
产出：可 Diff 的候选变更

05 / VERIFY
测试与审查
自动测试 · AutoReview · 回归
门禁：证据不足不进入提测

06 / RELEASE
提测、发布与回退
统一运行 · 版本包 · 回退脚本
产出：可访问提测与交付证据

控制点：上下文可追溯 · 变更可 Diff · 测试可复现 · 发布可回退 · 责任可签字

平台不替人做业务决定，而是把人的判断放在正确的门禁位置。

HUMAN CONTROL

生成越快，门禁越重要

AI负责候选，人负责边界与签字

立项 / 安全边界
范围、权限、数据与外部连接

业务 / 架构确认
业务规则、兼容性与长期可维护

发布 / 验收签字
上线、回退与责任确认

每个门禁留下四类证据
依据 · 差异 · 测试 · 版本

私有化部署 · 多模型适配
统一权限审计 · 可观测运行

接入现有系统，不把系统推到重来。

可见的AI研发：一段任务，四个证据面

需求上下文、可运行原型、审查控制、统一运行环境在同一工作区连续留痕；截图保留完整工作区并已脱敏。

01 / 需求上下文：文档、版本与AI助手的同一基线



同一需求文档里，需求修订、对话上下文与版本状态互相引用。

02 / 原型生成：从需求继续长成可运行页面



原型与代码变更在同一任务中继续迭代，不是结尾的一次性演示。

03 / AutoReview / Plan



审查项、修复计划与门禁成为研发动作的一部分。

04 / 运行提测



前端、服务、缓存、数据库与存储进入同一运行栈。

亿信华辰公开数据 · 研发侧口径

>70%

今年研发侧AI提效

仅代表研发侧效率，不泛化为客户收益、项目ROI或效果承诺。

截图来自EsXCode工作区；客户、项目与人员信息已脱敏。具体能力与效果以实际项目验证为准。

研发组织的分水岭：不是谁会用AI，而是谁先重写工作方式

我们把自研产品研发中的五条日常动作，沉淀为可复用的组织经验：让“个人更快”变成“团队可复现”。



当生成变便宜，真正被重估的是

四类能力决定AI产出能否变成组织交付

判断力 做什么、何时继续、何时停
从“执行任务”转向“定义问题与验收条件”

上下文 业务规则、历史决策、系统演进
不是谁的私人经验，而是可调用的组织资产

验证 影响、测试、安全、回退
生成越快，证据越要在流程里自动产生

系统设计 边界、接口、演进与可维护
AI能写代码，但不能替团队承担架构后果

角色与岗位不再按“谁写得多”分工

产品/业务更早做原型，研发更早参与场景判断

质量角色前移为规则、评测与门禁设计

边界会随模型能力移动，所以每季度重估一次：
哪些必须人判断，哪些可以交给AI。

组织转型不是“人人都买了AI”，而是“组织开始用AI重写成本结构”。

过去做办公：信息在生产，工作却在人之间搬运

制度、合同、Excel、报告、会议与审批分散在不同入口；真正耗时的是查、抄、核、改、等确认。

一张典型的金融办公“断点地图”

同一件事，被切成许多次手工接力



人成了上下文与流程之间的“手工总线”

工具越多，切换越多；信息越散，确认越慢。

租赁机构的日常	合同与投放材料 · 租后风险分析 · 资产分类 · 监管报送
证券机构的日常	研究与底稿 · 合规审查 · 客户服务 · 运营报表

断点不在“不会写”，而在这四件事

AI如果只做问答，仍然接不上完整工作

01 / 口径散

制度、产品、合同、指标解释不在同一处

答案有“样子”，却没有依据

02 / 数据难核

口径、来源、期间、版本靠人确认

数字一多，核对成本指数上升

03 / 材料重复写

同一事实被写进报告、汇报与审批

跨任务上下文无法继承

04 / 流程难交接

任务状态、确认意见与产物散落

换个人接手，等于重新问一遍

办公AI的起点，不是再开一个聊天入口，而是把知识、任务、系统与责任接回同一条工作链。

EsXWork：面向机构的AI办公平台

接住知识、文档、数据、会议、任务与系统，把办公从“生成一段内容”推进到“完成一项工作”。

工作从哪里来： 一句话任务　文件 / 邮件　会议　知识库　数据　业务系统事件 → **EsXWork 工作空间** → **可交付结果**

01 / KNOWLEDGE
企业知识与检索
制度、合同、项目、指标与历史决策
按权限检索，答案带出处与版本
交付物：带依据的答案与知识卡

02 / DOCUMENT
文档与材料
Word / Excel / PPT / PDF
按模板起草、修改、比对与渲染检查
交付物：可直接评审的材料包

03 / DATA
问数与分析
自然语言问数、口径解释、趋势与归因
图表、明细与数据来源一起交付
交付物：可复核的数据结论

04 / MEETING & TASK
会议、任务与看板
纪要、行动项、责任人、时限与进度
从信息整理推进到任务跟踪
交付物：可推进的任务对象

05 / RESEARCH
调研与专业报告
多源检索、提纲、比较、引用与结论
把“先搞懂”变成结构化的深度报告
交付物：有来源的调研与方案

06 / WORKFLOW
流程与人工卡点
表单、会签、确认、驳回与归档
AI推进流程，人保留关键决定权
交付物：可追溯的责任链

07 / AGENTS
多Agent与技能
专业分工、并行任务、统稿与验收
长任务异步推进，短任务可中断
交付物：可组合的专业产能

08 / CONNECTORS
连接企业与系统
API / MCP / OA / ERP / 数据平台
受控读取、写入与操作
交付物：从“会说”到“会办”

所有能力共用一层“可治理的工作空间”

个人记忆与偏好　项目上下文与版本　企业知识与图谱　权限 / 审计 / HITL / 私有化

公开产品形态可参照 WorkBuddy 类AI办公工作台与企业级AI助手；EsXWork按机构私有化、多模型与金融治理要求配置，能力以项目验证为准。

监管报送，是EsXWork上的专业空间： 高责任、长链路、强证据，正好检验办公平台能否真正进入金融生产。

监管报送不是从报表开始，而是从制度变化开始

一条制度变化要穿过识别、口径、模型、ETL、校验、报表、报文与评审；报表只是结果，证据还要反向回流。

正向链：制度下发以后，影响沿业务语义逐层传导

不是“再做一张表”，而是生成一组版本化变更

01 / 触发	02	03	04	05	06	07 / 结果	08	09
制度下发	**变更识别**	**口径**	**模型**	**ETL**	**校验规则**	**报表**	**报文**	**评审**
通知、办法、附件 明确报告义务与频率	新增、表样、说明 审核公式、停用	指标、范围、期间 解释与业务确认	实体、字段、映射 DDL与回退脚本	取数、清洗、编排 依赖与回退	完整性、勾稽 阈值与误报	表样、取数、勾稽 差异与说明	生成、结构校验 报表一致性	归档 发布
起点	A1	人审卡点	A3	A4	A5	不是起点	A7	A8

四类变化，四种影响面： 新增报表　表样变更　填报说明 / 审核公式　报表停用

反向链：执行端不是只交结果，还要把可复核的证据送回去

口径依据与变更记录 - - - - 模型映射与数据血缘 - - - - 测试 / 校验 / 差异 - - - - 版本与回退包 - - - - 评审意见与审计留痕

每一次变化都要回答：为什么改、改了什么、怎么验证、出了错如何退回、由谁确认。

新增报表

从制度义务到整套数据链

影响：口径、模型、ETL、规则、报表、报文

主链最容易漏掉的是“验收条件”

表样变更

行列、指标位置与取数方式变化

影响：报表、取数、勾稽、历史对比

必须同时检查跨表与跨期一致性

填报说明 / 审核公式

语义不变，解释与校验发生变化

影响：口径解释、规则、误报与人工复核

最容易被当作“小改动”低估

报表停用

依赖关系、历史归档与后续解释

影响：任务、校验、报表、归档与查询

删除之前，要先证明“谁还在用”

监管交付的复杂度，不在“一张表”，而在一组相互依赖、必须证明的变化。

一个专业工作台：对话、看板、卡点、证据围绕同一任务

这不是另一个平台，而是EsXWork在监管报送中的专业空间；以下内容来自当前研发与项目实践，能力以项目验证为准。

01 / 对话工作台

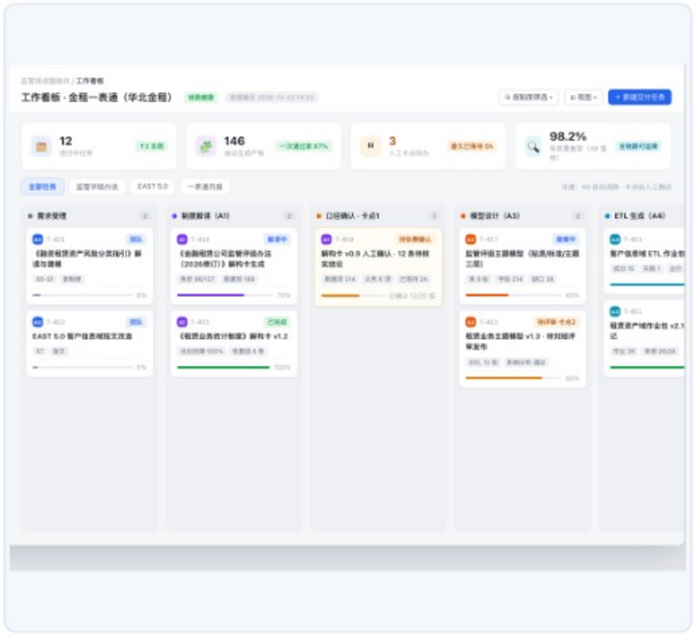
自然语言下发 · A0拆解 · 上下文连续



同一个会话里，能看到任务、执行轨迹与已产出物。

02 / 工作看板

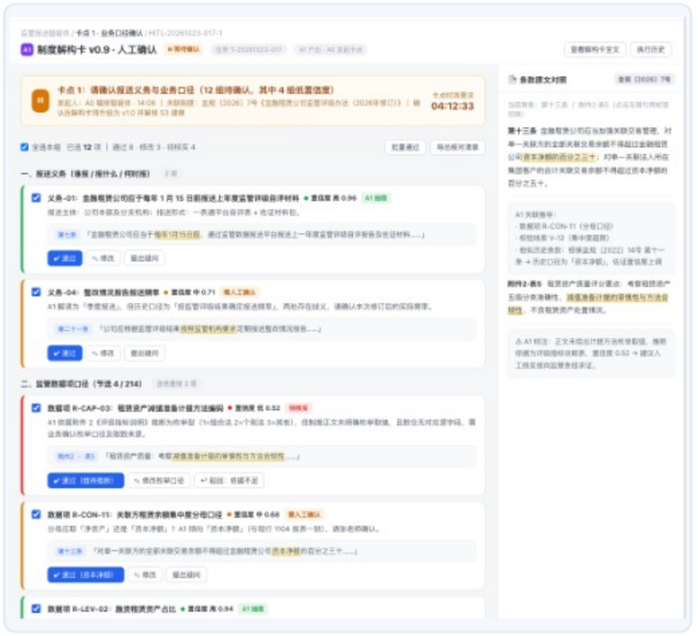
泳道推进 · 卡点高亮 · 一次通过率可见



进度不再是口头汇报：任务状态、产物数量与等待时长都可查。

03 / 人工确认卡点

置信度 · 原文对照 · 通过/修改/驳回留痕



卡点不是弹窗，而是带条款依据、修改记录与回退选项的专业确认。

同一任务对象贯穿五件事： 任务与阶段 · 执行证据 · 人工卡点 · 版本与回退 · 审计留痕　　不是四套系统拼接

界面来自研发与项目实践环境；信息已脱敏，能力以项目验证为准。

A1–A8：八个专业Agent，不是一个万能助手

A0负责拆解、调度和上下文；每个Agent只交付一种专业产物；三类人工卡点把责任留在流程里。

专业分工： 制度解读 · 问数 · 模型 · ETL · 校验 · 报表 · 报文 · 评审　　**人工卡点：** 业务口径 / 模型评审 / 发布确认



A1 · 制度解读 清单



A2 · 智能问数 口径/血缘/影响



A3 · 模型设计 ER/DDL/回退



A4 · ETL生成 SQL/编排/回退



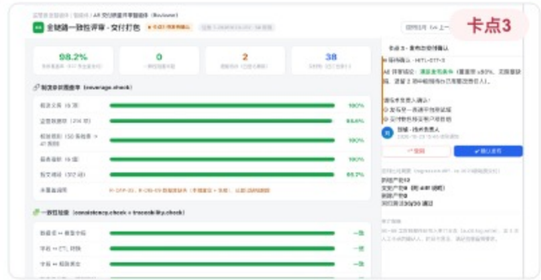
A5 · 校验规则 SQL



A6 · 报表设计 表样/取数/差异



A7 · 报文生成 XML/JSON/一致性



A8 · 质量评审 测试/证据/归档

A0 · 编排层　任务拆解 · 上下文装配 · Agent路由 · 卡点派发 · 失败恢复 · 审计留痕　一个任务，多专业协作

为什么不做“一个万能Agent”？ 专业任务需要专业上下文、工具与验收条件；拆开后，每一步可复用、可替换、可审查，失败也能局部回退。

界面与数字来自当前研发与项目实践环境；场景参考金租“一表通”，不代表所有机构；具体能力以项目验证为准。

一个新增报表的旅程：从一条制度条款，到一套可评审的交付

主线A1→A3→A4→A5→A7→A8；A2与A6按需侧向支持。每一步都有输入、产物、门禁与回退。

A1 · 制度解读
输入
制度原文 / 现行口径
产物
变更清单 / 口径草案
门禁
业务口径确认
回退：制度/口径基线

A3 · 模型设计
输入
确认口径 / 元数据血缘
产物
映射 / DDL / 回退脚本
门禁
数据模型评审
回退：DDL回退脚本

A4 · ETL生成
输入
模型映射 / 加工规范
产物
ETL / SQL / 依赖编排
门禁
数据作业评审
回退：ETL/编排版本

A5 · 校验规则
输入
规则清单 / 表结构
产物
校验SQL / 规则库
门禁
规则预期确认
回退：规则版本

A7 · 报文生成
输入
报表结果 / XSD
产物
报文 / 一致性证据
门禁
报验 / 一致性
回退：报文版本

A8 · 质量评审
输入
A1–A7全链交付物
产物
测试 / 评审 / 归档
门禁
安全 / 发布确认
回退：问题闭环清单

侧向支持 A2 · 智能问数
指标口径 · 血缘链路 · 历史变更 · 影响范围 · 数据异常解释
在A1口径澄清、A3模型映射、A5校验预期和A8评审时按需调用。
不改变主链产物与责任人，只补充解释证据。

侧向支持 A6 · 报表设计
表样 · 单元格取数 · 汇总 · 差异对比 · 报表开发规范
在A5规则确认、A7报文映射和A8一致性检查时按场景启用。
报表只是变化链的结果，不是起点；A6负责结果形态，不替代口径与模型。

A0编排贯穿整条旅程　任务拆解 · 上下文分发 · 产物交接 · 卡点催办 · 失败重试 · 版本归档

三道必过人审：　1 业务口径　2 模型评审　3 发布确认　每次通过、修改、驳回都进入审计轨迹。

新增报表、表样变更、填报说明调整、报表停用均可复用同一套“识别—确认—生成—验证—归档”结构；具体作业按场景编排。

AI把每一步变快，更把“每一步能否交接”变成可验证的工程问题。

信任但验证：AI做标准化，人守不可转移的责任

AI可以提高一致性和速度，但不能替代业务目的、风险判断、签字责任与组织后果。

AI / STANDARDIZE

把可规则化的工作做稳

快、全、一致，但仍是候选与辅助

01 结构化、摘要与初稿
02 规范、格式与明显缺陷检查
03 影响范围与异常线索初筛
04 测试用例、回归与证据整理
05 跨版本差异与一致性比对

AI的强项：高吞吐、可重复、不遗忘规则

同一套验证，不从“相信”开始

验证

证据优先

依据可追溯 · 结果可复现
异常可停止 · 版本可回退

验证后再交付

HUMAN / JUDGE

对目的、风险与后果负责

慢一点，但必须能解释与承担

01 业务规则是否成立
02 数据与模型假设是否合理
03 安全边界与外部影响
04 长期可维护、体验与组织后果
05 签字、发布、外部沟通

人的责任：目的、例外、边界与最终判断

跨研发与办公，三道“停下来想一想”的闸门

闸门 1 · 目标与口径

需求范围、业务口径、数据边界是否被正确理解?

典型：监管口径、系统改造范围、产品规则

闸门 2 · 方案与影响

架构、模型、接口与流程变化是否可解释、可维护?

典型：模型评审、系统影响分析、权限设计

闸门 3 · 发布与后果

高风险动作是否经过授权，异常能否停止与回退?

典型：资金、审批、删除、外部报送与发布

规则：AI可以发起候选，但不能绕过门禁；人能授权，也必须留下依据。

可停止 · 可解释 · 可回退 · 可追责

AI越强，人的工作越不是“少做”，而是“在更关键的位置做对”。

两个自研平台，一套可组合底座

共享运行、上下文、工具与治理；平台竞争从功能清单转向工程位置与接口。

EsXCode / AI研发
系统建设与持续演进
需求/变更/测试/回退

EsXWork / AI办公
知识、任务与专业交付
知识/数据/流程/归档

统一工作空间： Chat / API / 文件 · 任务会话 · 工程资产 · 证据 · 人工门禁 · 审计轨迹

同一任务对象贯穿研发与办公

从一键功能，到可组合的工程默认值

能力沉淀在接口与资产，越用越可复用

01 / CONTEXT & MEMORY
上下文与记忆
输入 / 历史 / 环境 / 业务数据
短期会话 + 长期记忆 + 分层装配
知识库 / 状态 / 版本 / 项目轨迹

02 / AGENT RUNTIME
Agent运行核心
感知 → 规划 → 推理 → 行动
Loop · 多Agent · 状态机 · 失败恢复
可替换 / 可治理 / 可停止

03 / MODEL ROUTER
模型路由与适配
按任务复杂度与成本选择模型
多模型接入，不绑定单一厂商
国产模型友好 / 私有模型端点

04 / TOOLS
工具与执行
MCP / API / 代码 / 文件
授权边界内读、写、执行
从"会说"到"会办"

05 / RAG
知识与检索
向量 / 混合 / 图谱 / 结构化
知识可检索、可引用、可追溯
RAG / 影响分析 / 语义检索

06 / SAFETY
安全、权限与治理
身份 / 脱敏 / 内容安全 / 合规
高风险动作拦截，人工确认可审计
审批 / 驳回 / 修改 / 审计留痕

07 / OBSERVABILITY & EVALS
观测、评测与学习
Trace / 指标 / 评测 / 告警
反馈沉淀为规则、技能与策略优化
证据包 / 回归 / A-B 实验

08 / ARTIFACT & RECOVERY
产物版本与恢复
需求、原型、代码、文档、报表与报文
统一版本、差异、回退与交付归档
可恢复 / 可复用 / 可移交

基础设施与部署 私有化 / 专有云 · 容器与安全沙箱 · 存储 / 数据库 / 向量检索 · 企业身份与审计 · 多模型网关 · 企业系统

平台比较不只看功能表，更看运行、接口、证据与生态位置；本页为工程拆解，不代表同架构或效果等同。

功能会被复制，模型会被替换；可复用的工程资产，才会持续复利。

亿信华辰
ESENSOFT

24 / RETURN TO THE QUESTION

20周年客户大会 · 主题分享

20
YEARS

回到开场那个问题

如果AI Agent让系统上线与监管报送都快了一倍，贵机构真正变难的，会是什么？

不是多几个工具，而是：选对问题、接全上下文、证明可信、守住责任。

01 / CHOOSE
选对问题
高频、高价值、边界清晰，
并且可以被验收。

02 / CONNECT
接全上下文
知识、数据、系统、流程与历史，
进入同一条任务链。

03 / VERIFY
证明可信
影响分析、测试、评测与版本，
在过程里自动产生。

04 / OWN
守住责任
人负责目标、例外、风险与签字；
AI负责推进与证据。

从一个系统开始，先做三件事

01 接一个系统 选一个有明确边界的存量模块或新建场景。 → **02 改一段流程** 从知识、材料或口径的一个真实断点开始。 → **03 留一组证据**

把选择、变化、验证、版本与责任留成可复核的交付，而不是一次性的演示。

从“更快一点”，走向可复用的组织能力。

因数而生　共创智能时代

亿信华辰解决方案专家　吴思全　|　2026.10.30